Name: P.Jeeva
Reg: no: 192412636

CO4-AT2-INDUSTRY PROBLEM SOLVING

Question 1

Data Plan Recommendation Using K-Nearest Neighbour / Case-Based Learning

1.1 Problem Overview

A telecommunications company wants to recommend data plans (e.g., basic, standard, premium, unlimited-streaming) that best match each customer's actual usage behaviour. Because customer needs are highly individual and historical outcomes (which plan satisfied which type of user) are readily available, this is a natural fit for instance-based learning: rather than building one global rule, the system reasons directly from similar past customers to the current one.

Two closely related instance-based techniques are proposed together: K-Nearest Neighbour (KNN) supplies the similarity search and voting mechanism, while Case-Based Reasoning (CBR) supplies the broader lifecycle — retrieving, reusing, revising, and retaining cases — so that the recommendation system keeps learning as usage patterns evolve.

1.2 System Architecture

Each customer is represented as a feature vector built from their usage logs. New customers (or existing customers being re-evaluated) are compared against a case base of past customer profiles that already have a known, satisfactory plan assignment. The closest matches vote on the recommended plan.

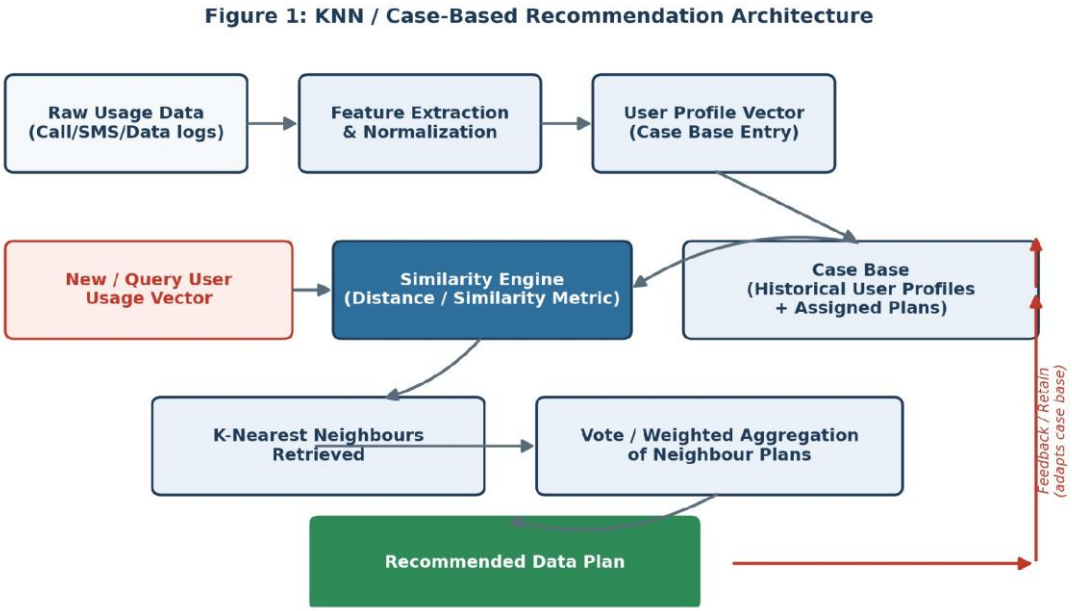

Figure 1: End-to-end KNN / Case-Based recommendation pipeline, including the feedback loop that lets the case base adapt over time.

1.3 Feature Representation

Raw call detail records, data-usage logs, and billing history are aggregated into a fixed-length numeric profile per customer. Typical features:

Feature	Description	Example Value
Avg. Monthly Data (GB)	Mean data consumed per billing cycle	14.2 GB
Peak-Hour Usage Ratio	Fraction of data used 6pm–11pm	0.42
Voice Minutes / Month	Average outgoing + incoming call minutes	180 min
SMS Count / Month	Average text messages sent	35
Streaming App Share	% of data from video/music apps	58%

Roaming Frequency	Trips per year requiring roaming data	3

Feature	Description	Example Value
Device Type	Smartphone tier / OS (categorical, encoded)	High-end Android
Bill Sensitivity	Historical plan downgrades / complaints	Low

Because features are on very different scales (GB vs. minutes vs. ratios), min-max normalization or z-score standardization is applied to every feature before distance is computed, so that no single attribute (e.g., voice minutes with large raw values) dominates the similarity score purely due to scale.

1.4 Measuring Similarity Between Users

For a new customer query vector q and a stored case x, similarity is computed using a distance metric on the normalized feature vector. The most common choices:

Metric	Formula (conceptual)	When to use
Euclidean Distance	$\text{sqrt}(\Sigma\ (q\_i - x\_i)^2)$	Continuous numeric features, no strong outliers
Manhattan Distance	$\Sigma\ \lvert q\_i - x\_i \rvert$	Robust to outliers, mixed-magnitude features

Cosine Similarity	$(q \cdot x) / (\|\|q\|\| \cdot \|\|x\|\|)$	Usage "shape" matters more than absolute volume
Weighted Euclidean	$\text{sqrt}(\Sigma\ w\_i\ (q\_i - x\_i)^2)$	Some features (e.g., data usage) matter more than others

In practice, weighted Euclidean distance is preferred: business rules assign larger weights (w_i) to features that most strongly separate plan types (e.g., data volume, streaming share), and smaller weights to weaker signals (e.g., SMS count). Categorical features such as device type are handled with simple matching distance (0 if equal, 1 if different) and folded into the same weighted sum.

Worked Example — Distance Calculation

Suppose a new customer's normalized vector is compared against three stored cases on two illustrative features (data usage, streaming share):

Customer (Case)	Data Usage (norm.)	Streaming Share (norm.)	Distance to Query	Assigned Plan
Query User	0.72	0.65	—	?
Case A	0.70	0.60	0.054	Premium Streaming
Case B	0.30	0.20	0.573	Basic
Case C	0.75	0.55	0.104	Premium Streaming

Case D	0.68	0.70	0.064	Premium Streaming

With k = 3, the nearest neighbours are Case A, Case D, and Case C — all previously assigned the "Premium Streaming" plan — so the system recommends that plan to the query user by unanimous vote.

1.5 Generating the Recommendation

- Retrieve the k nearest cases to the query vector using the chosen distance metric (k is typically tuned between 5 and 15 via cross-validation on historical satisfaction outcomes).
- Aggregate their plan labels:
  - Majority vote for classification-style recommendation (which single plan to offer).
  - Distance-weighted vote (closer neighbours count more, weight = 1 / distance) to reduce the influence of borderline matches.
- Apply business constraints as a revision step (e.g., do not recommend a plan the customer is already on; respect minimum contract eligibility).
- Present the top recommendation plus 1–2 close alternatives so the customer/agent has options, ranked by neighbour vote strength.

1.6 Adapting to Changing User Behaviour (CBR Lifecycle)

Usage habits drift over time — a customer who once mostly used voice/SMS may become a heavy streamer, or seasonal patterns may shift usage. The system stays current using the four-step Case-Based Reasoning cycle:

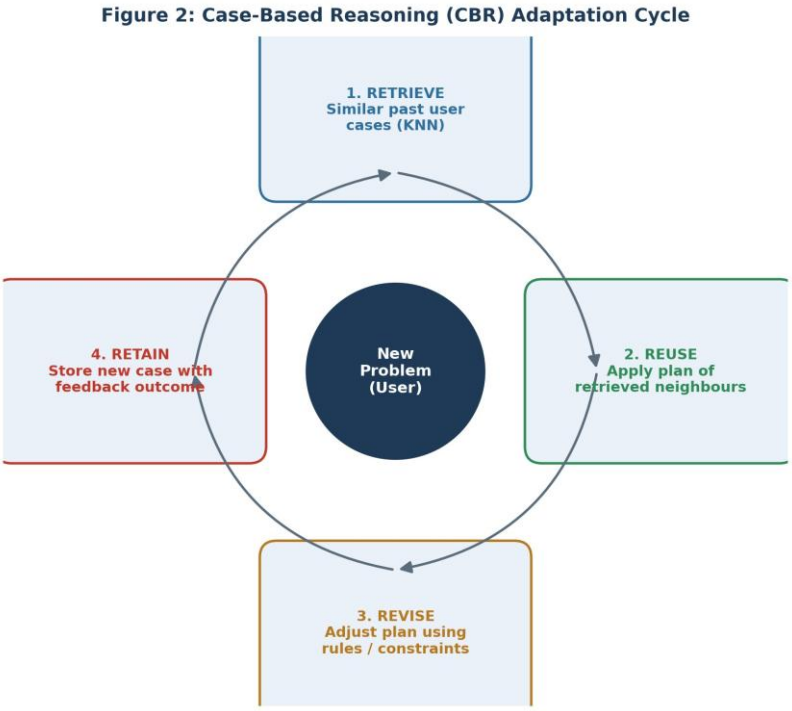

Figure 2: The Retrieve → Reuse → Revise → Retain cycle keeps the case base current.

Mechanism	How it Handles Drift
Sliding-window profiles	Usage vectors are recomputed monthly from a rolling 3–6 month window, so old behaviour naturally ages out of the similarity calculation.
Case retention ("Retain")	Every resolved recommendation (accepted, rejected, or later changed by the customer) is added back into the case base with its real outcome, so the case base grows and reflects current trends.
Case forgetting / pruning	Very old or rarely-matched cases are periodically archived to keep the case base efficient and representative of current usage norms.

Dynamic re-weighting	Feature weights (w_i) are periodically re-tuned as new data shows which attributes best predict satisfaction, so the notion of "similarity" itself evolves.
Re-triggering	A significant change in a customer's own monthly profile (e.g., data usage jumps 50%) automatically re-triggers a fresh KNN lookup and new recommendation.

This closed feedback loop is what distinguishes the approach from a static classifier: the model has no fixed training phase — it is a living, continuously updated case base that automatically reflects the telecom company's most recent customer behaviour without manual retraining.

1.7 Advantages and Limitations

Aspect	Advantage	Limitation
Transparency	Recommendation is explainable ("customers like you chose this plan")	—
Aspect	**Advantage**	**Limitation**
Adaptability	New cases improve the system immediately, no retraining cycle needed	Case base can grow very large, requiring indexing (e.g., k-d trees, ball trees) for fast lookup
Cold start	Works even with a modest number of cases	New customers with unusual usage patterns may have no close neighbours

Maintenance	No complex model tuning beyond k and feature weights	Sensitive to irrelevant/noisy features unless weighting is well-tuned

Question 2

Electricity Consumption Prediction Using Locally Weighted Regression / RBF Networks

2.1 Problem Overview

An energy management company needs to predict electricity consumption from factors such as time of day, day type, weather (temperature, humidity), and appliance usage state. Consumption is highly non-linear and context-dependent: a household's load curve on a hot weekday afternoon looks nothing like a mild weekend evening. A single global regression line cannot capture this, so the solution uses local learning — methods that fit the model's behaviour to the neighbourhood of each specific query point rather than the whole dataset at once.

Two complementary local-learning techniques are proposed: Locally Weighted Regression (LWR), which fits a small weighted regression around each query on demand, and RBF (Radial Basis Function) Networks, which pre-compile many such local regions into a trained network for fast repeated predictions. In practice, LWR is well suited to exploratory/offline analysis and small-to-medium datasets, while an RBF network is deployed for fast, real-time consumption forecasting.

2.2 Locally Weighted Regression (LWR)

LWR predicts the output for a query point x_q by fitting a fresh, simple regression (usually linear) using only training points near x_q, weighted by their distance to x_q. Points far from the query barely influence the fit; points close to it dominate.

The weight for each training point x_i is typically a Gaussian kernel:

$$w_i = exp(- ||x_i - x_q||^2 / (2\tau^2))$$

where τ (the bandwidth) controls how "local" the fit is. A small τ makes the model sensitive to fine local variation (e.g., a sudden appliance spike); a large τ smooths over noise but risks blurring genuine local patterns. The weighted points are then used to solve a weighted least-squares regression, and only the

prediction at x_q is kept — the whole process repeats independently for the next query.

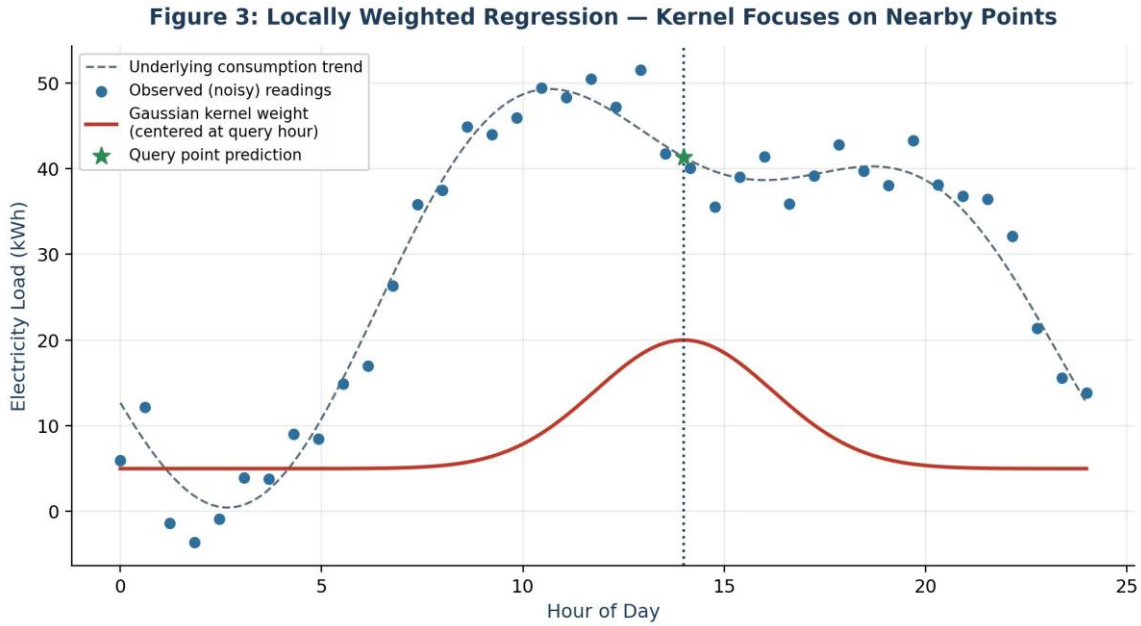

Figure 3: The Gaussian kernel (red) gives near-query hours high influence and distant hours almost none, letting the local fit track the true daily load curve through noisy readings.

Why Local Learning Captures Consumption Variation

- Time-of-day non-linearity: morning ramp-up, midday plateau, and evening peak each have different slopes; a local fit around "7pm" is not forced to also explain "3am" behaviour.
- Weather interaction effects: on hot days, air-conditioning dominates load in a way that a global linear model would average away; a local neighbourhood of "hot afternoons" captures that relationship directly.
- Appliance-state clustering: households with similar concurrent appliance states (e.g., washing machine + oven running) form natural local neighbourhoods with their own load behaviour.

2.3 RBF (Radial Basis Function) Networks

An RBF network formalizes the same "local influence" idea into a trainable neural architecture with three layers: an input layer for the context features, a hidden layer of RBF neurons (typically Gaussian), and a linear output layer.

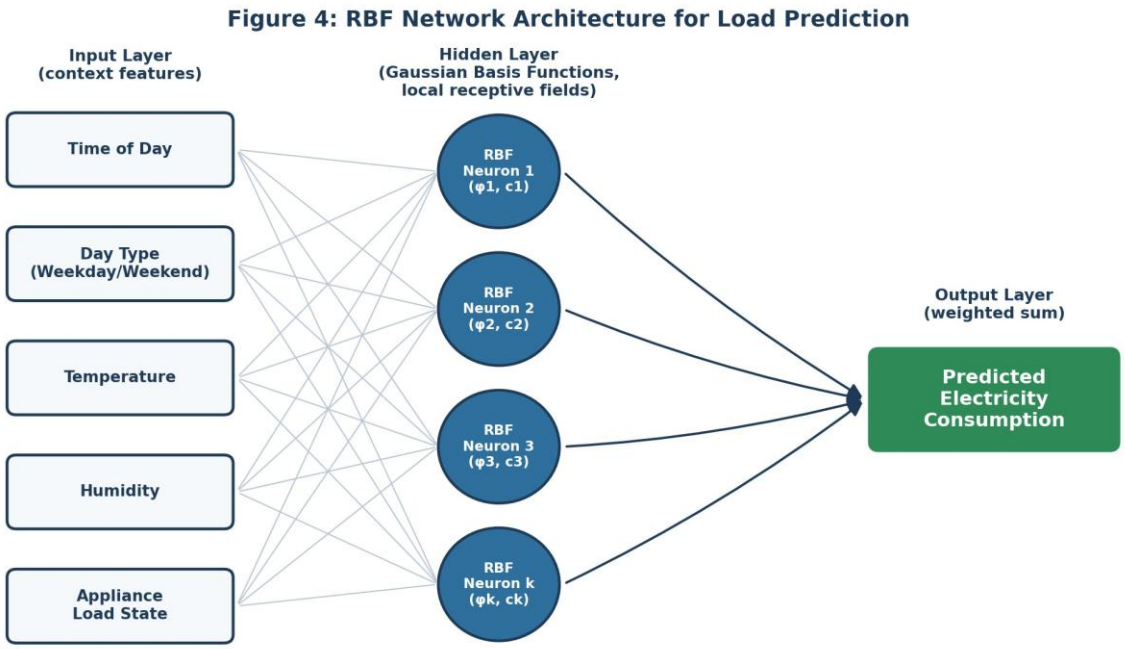

Figure 4: Each hidden neuron acts as a local receptive field centred on a prototype context (e.g., "hot, weekday, afternoon"); the output layer combines their activations linearly.

- Input layer: the feature vector (time of day, day type, temperature, humidity, appliance load state).
- Hidden (RBF) layer: each neuron j has a centre c_j (a prototype context) and computes $\varphi_j(x) = \exp(-\|x - c_j\|^2 / (2\sigma_j^2))$ — its activation is high only when the input is close to its centre, giving each neuron a local receptive field.
- Output layer: predicted consumption = $\Sigma (v_j \cdot \varphi_j(x))$, a linear combination of the local activations, where v_j are learned output weights. Training typically proceeds in two stages: (1) unsupervised clustering (e.g., k-means) to place centres c_j at representative regions of the feature space (so common contexts like "weekday morning, mild weather" get dedicated neurons), and (2) supervised least-squares or gradient-based tuning of the output weights v_j (and optionally the widths σ_j) against historical consumption readings.

2.4 Handling Noisy Data

Technique	Effect on Noise
Kernel weighting (LWR)	Distance-based weights naturally down-weight noisy outlier readings that happen to sit far from the local neighbourhood mean, since a point's influence decays smoothly with distance rather than being all-or-nothing.
Wider bandwidth / σ	Increasing τ (LWR) or σ_j (RBF) smooths over sensor noise and short random spikes at the cost of some local sharpness — a tunable bias/variance trade-off.
Regularized least squares	Ridge-style regularization in the weighted or RBF output regression prevents the model from over-fitting to noisy individual readings.
Redundant/averaged centres (RBF)	Multiple overlapping RBF neurons around similar contexts let the output layer average out random measurement noise rather than depending on one exact match.

2.5 Handling Seasonal Data

Technique	Effect on Seasonality
Cyclical feature encoding	Time-of-day and day-of-year are encoded as sin/cos pairs so “11:59pm” and “12:01am” are recognized as neighbours instead of being maximally distant.

Season-specific centres (RBF)	Separate clusters of RBF centres naturally form for summer cooling-load patterns vs. winter heating-load patterns, since k-means placement is driven by the actual seasonal data distribution.
Seasonal bandwidth tuning (LWR)	τ can be widened during stable seasons and narrowed during rapidly transitioning ones (e.g., spring), keeping local fits appropriately responsive.

Technique	Effect on Seasonality
Periodic retraining	RBF centres and weights are refreshed each season (or via a rolling window) so the model tracks slow seasonal drift in baseline consumption, similar in spirit to the case-base retention idea in Q1.

2.6 LWR vs. RBF Networks — Comparison

Aspect	Locally Weighted Regression (LWR)	RBF Network
Learning style	Lazy (no explicit training phase; computed per query)	Eager (centres and weights trained in advance)
Prediction speed	Slower — refits a regression for every query	Fast — single forward pass through trained network
Memory	Must retain full (or windowed) training dataset	Only needs learned centres and weights

Best fit	Small/medium datasets, offline analysis, quick prototyping	Large-scale, real-time / repeated forecasting
Adaptability to drift	Naturally adaptive if the stored dataset window is updated	Requires periodic retraining or online weight updates
Interpretability	High — each prediction traces to nearby real observations	Moderate — centres are interpretable prototypes

2.7 Putting It Together — Deployment View

- Offline / analytical use (diagnosing unusual consumption, small utility pilots): use LWR directly against the historical readings database.
- Production forecasting (dashboards, demand response, billing prediction at scale): train an RBF network periodically (e.g., weekly) on the latest windowed data and serve predictions from the trained network for speed.
- Both approaches share the same core principle from Question 1's case-based system: predictions/recommendations come from what happened in similar past contexts, and the model stays accurate by continually refreshing its notion of "nearby" as new, real consumption and usage data arrives.